Module 4
Structure and File

4.1 Structure

- A structure is a user-defined data type that can be used to group items of possibly different types into a single type.
- The *struct* keyword is used to define a structure.
- The items in the structure are called its members and they can be of any valid data type.
- Example: A "student" has a name, rollNo, and marks. Instead of keeping separate variables, we make one structure.

4.1.1 Syntax of Structure

- There are two steps of creating a structure in C:
  1. Structure Definition
  2. Creating Structure Variables
- *Structure Definition*
  - A structure is defined using the *struct* keyword followed by the structure name and its members.
  - It is also called a structure template or structure prototype, and no memory is allocated to the structure in the declaration.

```
struct structure_name {
   data_type1 member1;
   data_type2 member2;
   ...
};
```

Where, structure_name: Name of the structure.

member1, member2, … : Name of the members.

data_type1, data_type2, … : Type of the members.

- Example:

```
struct Student {
   char name[50];
   int rollNo;
   float marks;
};
```

- *Creating Structure Variable*
    - After the structure definition, we have to create a variable of that structure to use it.
    - It is similar to the any other type of variable declaration:

      struct *structure_name var;*

    - Example:

```
struct Student s1;
```

    - We can also declare structure variables with structure definition.

      struct *structure_name {*
      ...
      }var1, var2....;

    - Example:

```
struct Student {
   char name[50];
   int rollNo;
   float marks;
}s1;
```

4.1.2 Initialisation

- Initialisation of the structure variable can be done when you declare the variable.
- Example:

```
struct Student s1 = {"John", 10, 87.5};
```

- After declaration, initialisation can be done by assigning values to each member. declared the struct:

```
struct Student s2;
strcpy(s2.name, "Alan");  // strings copied using strcpy
s2.rollNo = 15;
s2.marks = 95.0;
```

4.1.3 Accessing Members

- To access or modify members of a structure, use the dot operator '`.`':

structure_name . member1;

- Example:

  s1.rollNo = 10;

  scanf(“%f”, &s1.marks); // reading values from keyboard

  // Print values of structure variables

  printf("%s\n", s1.name);

  printf("%d\n", s1.rollNo);

- Program to create a structure to represent a student with name, roll no and marks as members:

```
#include <stdio.h>
// Defining a structure to represent a student
struct Student {
   char name[50];
   int age;
   float grade;
};

void main() {
    // Declaring and initializing a structure variable
     struct Student s1;
     printf("Enter the name of student:");
     scanf("%s",s1.name);
     printf("Enter the roll no. of student:");
     scanf("%d",&s1.rno);
     printf("Enter the name of student:");
     scanf("%f",&s1.marks);
     printf("Student Details\n");
      printf("Name\tRoll No\t\tMarks\n");
   // Accessing structure members
   printf("%s\t\t%d\t\t%f\n", s1.name, s1.rno, s1.marks);

  }
```

Output

Enter the name of student:Arun
Enter the roll no. of student:4
Enter the name of student:87.5
Student Details

Name	Roll No	Marks
Arun	4	87.500000

4.2 Array of Structure

- An array of structure is an array whose elements are structures.
- If you've defined a struct, you can store multiple "records" of that struct in an array.
- It can be considered as a table:
- For example: if a Student is a structure of array
-

Roll No	Name	Marks
5	John	52
10	Seena	60

- Each row is a structure, and the whole table is an array of structures.
- *Define the Structure*

```
struct Student {
  int rollNo;
  char name[50];
   float marks;
};
```

- *Declare an Array of that Structure:*

```
struct Student s[3];  // array of 3 Student structures
```

- This allocates memory for 3 struct Student objects.

students[0] is the first student.
students[1] is the second.
students[2] is the third.

- Input and output can be done using loops.
- Example: Program to create an array of structure for 3 students

```
#include <stdio.h>
#include <string.h>

struct Student {
   char name[50];
   int rollNo;
   float marks;
};

void main() {
   struct Student s[3];   // array of structures

   for (int i = 0; i < 3; i++) {
      printf("\nEnter details for student %d:\n", i + 1);
      printf("Name: ");
      scanf("%s", s[i].name);   // or use fgets for full name
      printf("Roll No: ");
      scanf("%d", &s[i].rollNo);
      printf("Marks: ");
      scanf("%f", &s[i].marks);
   }

   printf("\n Student Details\n");
   for (int i = 0; i < 3; i++) {
      printf("\nStudent %d:\n", i + 1);
      printf("Name: %s\n", s[i].name);
      printf("Roll No: %d\n", s[i].rollNo);
      printf("Marks: %.2f\n", s[i].marks);
   }
 }
```

4.3 Structure and Pointers (Pointers to Structures)

- A pointer to a structure is just a pointer variable that stores the address of a structure variable.
- If a pointer to structure is used, the structure's members can be accessed using the arrow operator (->).
- Example:

```
struct Student {
    char name[50];
    int rollNo;
    float marks;
};
```

- *Declaring a Structure and a Pointer:*

```
struct Student s1;        // a structure variable
struct Student *ptr;      // a pointer to struct Student
ptr = &s1;                // store address of s1 in ptr
```

- Accessing Members Using Pointer

```
ptr->rollNo = 10;
strcpy(ptr->name, "John");
ptr->marks = 87.5;
```

- Example:

```
#include <stdio.h>
#include <string.h>
struct Student {
    char name[50];
    int rollNo;
    float marks;
};

void main() {
    struct Student s1;
    struct Student *ptr;

    ptr = &s1; // pointer points to s1

    // taking input using pointer
    printf("Enter name: ");
    scanf("%s", ptr->name);          // read string into name
    printf("Enter roll number: ");
    scanf("%d", &ptr->rollNo);         // read integer
```

```
    printf("Enter marks: ");
    scanf("%f", &ptr->marks);          // read float
    // printing using pointer
    printf("\nName: %s\n", ptr->name);
    printf("Roll No: %d\n", ptr->rollNo);
    printf("Marks: %.2f\n", ptr->marks);
}
```

4.4 Passing Structure as Parameter to Functions

- A structure can be passed to a function in two ways:
    - Pass Structure by Value
        - You pass the whole structure.
        - The function gets a copy.
        - Any changes inside the function do not affect the original.
        - Example:

```
#include <stdio.h>
#include <string.h>
struct Student {
    char name[50];
    int rollNo;
    float marks;
};
// function takes a struct as value (copy)
void display(struct Student s) {
    printf("\nName: %s", s.name);
    printf("\nRoll No: %d", s.rollNo);
    printf("\nMarks: %.f\n", s.marks);
}
void main() {
    struct Student s1 = {"John", 10, 87.5};
    display(s1);  // pass whole struct
    // s1 is unchanged after display()
 }
```

- Pass Structure by Reference (Pointer)
  - We can pass the address of the structure.
  - The function receives a pointer, so it can directly access (and modify) the original struct.
  - Example:

```
#include <stdio.h>
#include <string.h>
struct Student {
   char name[50];
   int rollNo;
   float marks;
};
// function takes a pointer to struct
void input(struct Student *s) {
   printf("Enter name: ");
   scanf("%s", s->name);       // arrow operator
   printf("Enter roll no: ");
   scanf("%d", &s->rollNo);
   printf("Enter marks: ");
   scanf("%f", &s->marks);
}

// function takes a pointer to struct (read-only)
void display( struct Student *s) {
   printf("\nName: %s", s->name);
   printf("\nRoll No: %d", s->rollNo);
   printf("\nMarks: %f\n", s->marks);
}
void main() {
   struct Student s1;
   input(&s1);     // pass address so function fills s1
   display(&s1);   // pass address to display s1
}
```

4.5 Union

- A union is a user-defined data type that allows different data members to share the same memory location.
- It is similar to a struct, but:
    - In a structure, each member has its own storage.
    - In a union, all members share the same memory, so only one member can hold a valid value at a time.
- Syntax:

```
union UnionName {
   data_type member1;
   data_type member2;
   data_type member3;
};
```

- Declaring variables:

```
union UnionName var1, var2;
```

- Example:

```
union Data {
   int i;
   float f;
   char str[20];
};
```

- Memory Size of a Union:

```
union Data {
   int i;      // 4 bytes
   float f;    // 4 bytes
   char str[20]; // 20 bytes
};
```

  Therefore, size of union = 20 bytes (size of largest member).

- Difference between Structure and Union

Feature	Structure	Union
Definition	A structure is a user-defined data type that groups different data types into a single entity.	A union is a user-defined data type that allows storing different data types at the same memory location.

Keyword	Defined using the *struct* keyword.	Defined using the *union* keyword.
Memory Allocation	Each member gets its own memory.	All members share the same memory.
Size	The size is the sum of the sizes of all members, with padding if necessary. Example: struct S { int a; float b; char c; }; Size ≈ 4 + 4 + 1 = 12 bytes	The size is equal to the size of the largest member, with possible padding. Example: union U { int a; float b; char c; }; Size = largest member = 4 bytes.

4.6 Enumerations

- An enum is a user-defined type consisting of a set of named integer constants.
- By default, the first name has value 0, the next 1, and so on (unless specified manually).
- This makes the program more readable and maintainable.
- Syntax:

```
enum EnumName {
   name1,
   name2,
   name3 = 5,  // can assign custom value
   name4       // automatically becomes 6
};
```

- Example:

```
enum Weekday {Mon, Tue, Wed, Thu, Fri, Sat, Sun};
void main() {
   enum Weekday today;
   today = Wed;
   if (today == Wed)
      printf("It's Wednesday!\n");
   printf("Value of Wed = %d\n", Wed);        // prints 2 (Mon=0,Tue=1,Wed=2)
 }
```

4.7 Files

- A file is a named area in storage (like hard disk) where data is kept permanently.
- Using files, a program can store data permanently, unlike variables which lose values once the program ends.

4.7.1 File Handling

- File handling allows programs to interact with files stored on a persistent storage device, enabling data to be saved and retrieved even after the program terminates.
- This is achieved through a set of standard library functions defined in stdio.h.
- The steps involved in file processing are:
    1. Declare a file pointer: A FILE pointer, declared as FILE *fp;, is used to reference a file and maintain its current position during operations.
       Example:
       ```
       FILE *fp;
       ```
    2. Open the file: The function fopen() is used to open a file with the correct mode.
       Syntax:
       ```
       fopen(filename, mode);
       ```
       Example:
       ```
       fp = fopen("data.txt", "r"); //opens a file named data.txt in read mode
       ```
    - The mode tells the compiler how the file should be handled (read, write, append, etc.). The different modes are:
        - **“r”(Read)** - Opens an existing file. If the file does not exist, the opening fails. The file pointer starts at the beginning of the file.
        - **"w" (write)** - Opens a file to write only.If the file already exists, its content is erased. If the file does not exist, a new one is created.The file pointer starts at the beginning of the file.
        - **"a" (append) -** Opens a file to write only, but writing always happens at the end of the file.Old data is not erased. If the file does not exist, it is created. The file pointer is positioned at the end.
        - **"r+" (read + write)** - Opens an existing file for both reading and writing. The file must already exist. Content is not erased. The file pointer starts at the beginning, so you can read and also overwrite parts if you want.
        - **"w+" (write + read)** - Opens a file for reading and writing. If the file exists, its content is erased immediately. If it does not exist, a new file is created. The file pointer starts at the beginning of the file.
        - **"a+" (append + read)** - Opens a file for reading and writing. If the file does not exist, it is created. Writing always goes to the end of the file (like "a"), but you can still read from anywhere in the file. The file pointer starts at the end for writing, but you can move it for reading.

3. Process the file: Read or write data using functions like fprintf(), fscanf(), fgetc(), fgets(), fputc(), fputs(), or binary functions like fread()/fwrite().
4. Close the file: Use fclose() to release the file and ensure all data is saved.
   Syntax:
   fclose(file pointer);
   Example:
   fclose(fp);

4.7.2 Input and Output Operations on Sequential Files

- C provides various functions for performing input and output operations with files.
- These functions are found in the <stdio.h> header file.
- ***Writing Data to a File:***
  - ***fputc():*** Writes a single character to a file.

    Example:

    fputc('A', fp);

    //writes the character 'A' to the file that is associated with the file pointer fp.
  - ***fputs():*** Writes a string to a file.
    Example:
    fputs("Hello World!", fp);
  - ***fprintf():***Writes formatted output to a file.
    Example:
    fprintf(fp, "Name: %s Age: %d\n", name, age);
  - ***fwrite() :*** Write to a Binary File
    Syntax: fwrite(data_buffer, size_of_element, number_of_elements, fp);
- ***Reading from a File:***
  - ***fgetc():*** Reads a single character from a file.
    Example:
    ch = fgetc(fp);
  - ***fgets():*** Reads a string from a file.
    Example:
    fgets(line, 100, fp);
  - ***fscanf():*** Reads formatted data from a file.
    Example:
    fscanf(fp, "%s %d", name, &age);
  - ***fread():*** Reads blocks of binary data from a file into memory.
    Syntax:
    fread(data_buffer,size_of_element, number_of_elements, fp);

- Sample Program: Find the number of characters in a file

```
#include <stdio.h>
void main() {
   FILE *fp;
   int ch;     // must be int, not char
   int count = 0;

   fp = fopen("input.txt", "r");
   if (fp == NULL) {
      printf("Error opening file!\n");
      return 1;
   }
   while ((ch = fgetc(fp)) != EOF) {
      count++;
   }

   printf("Total number of characters in the file: %d\n", count);

   fclose(fp);
}
```

4.8 Command Line Arguments

- Command line arguments are values passed to the main() function when a C program is executed from the command prompt (or terminal).
- They allow users to provide input directly when running the program.
- Syntax of main() with Command Line Arguments:

```
int main(int argc, char *argv[])
```

Where,

argc: Argument Count — number of arguments passed (including program name).

argv[]: Argument Vector — array of strings (character pointers) containing the arguments.

- Example: If you run this command in terminal:

```
./a.out hello world
```

- Then:

```
argc = 3
argv[0] = "./a.out"
argv[1] = "hello"
argv[2] = "world"
```

- Sample Program:

```
#include <stdio.h>
int main(int argc, char *argv[]) {
   printf("Number of arguments: %d\n", argc);

   for (int i = 0; i < argc; i++) {
      printf("Argument %d: %s\n", i, argv[i]);
   }
   return 0;
}
```

If run as:

```
./a.out apple banana cherry
```

Output will be:

```
Number of arguments: 4
Argument 0: ./a.out
Argument 1: apple
Argument 2: banana
Argument 3: cherry
```